

01 – Frontend Architecture

Zurück:

- [00 System Overview](#)

Weiter:

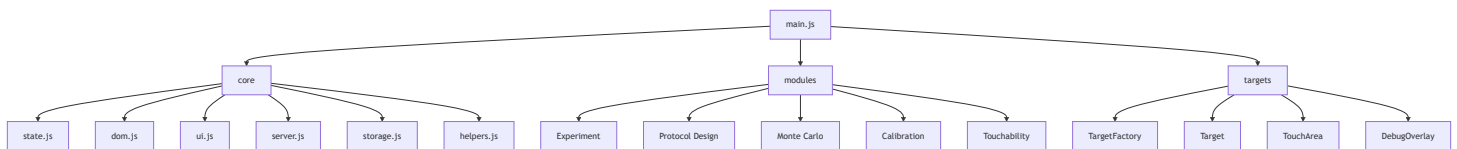
- [02 Experiment Runtime](#)

Ziel

Dieses Dokument beschreibt die Frontend-Architektur der Anwendung.

Nach dem Refactoring ist das Frontend in vier Ebenen organisiert:

```
main.js
  ↓
core/
  ↓
modules/
  ↓
targets/
```



1. main.js

Verantwortung:

- Anwendung starten
- Module initialisieren
- Event Handler verbinden
- Kalibrierung laden
- Touchability laden

Wichtig:

`main.js` enthält keine Fachlogik.

Es ist ausschließlich ein Bootstrap-Controller.

2. Core Layer

Ordner:

`static/javascript/core`

state.js

Speichert den globalen Zustand.

Beispiele:

`mmPerPx`
`sessionBlocks`
`currentProtocol`
`savedToPC`
`touchDiameterPx`

dom.js

Sammelt DOM-Referenzen.

Beispiel:

```
dom.buttonStart  
dom.btnSaveProtocol  
dom.callRect
```

ui.js

Verantwortlich für:

- Anzeigen von Screens
- HUD Updates
- Statusanzeigen

server.js

Kommunikation mit Flask.

Beispiele:

```
save_results  
api/protocols  
delete_protocol
```

storage.js

Frontend-Fassade für LocalStorage.

Delegiert an:

```
storage/  
├─ calibrationStorage.js  
├─ protocolStorage.js  
├─ touchabilityStorage.js  
└─ deviceSignature.js
```

helpers.js

Kleine Hilfsfunktionen.

Beispiele:

```
Fullscreen  
Orientation Lock  
Clamp  
Utility Funktionen
```

3. Module Layer

Ordner:

```
static/javascript/modules
```

Jedes Modul kapselt einen klaren Anwendungsbereich.

Calibration

calibration.js
calibrationHandlers.js

calibration/
├─ calibrationMath.js
└─ calibrationGestures.js

Verantwortung:

- Bildschirmkalibrierung
- mm/px Berechnung
- Gestensteuerung

Touchability

fingerTouchability.js
touchabilityRuntime.js
touchabilityHandlers.js

Verantwortung:

- Fingergrößenmessung
- Fallbackwerte
- Wmin Berechnung

Protocol Design

protocol.js
protocolDesignHandlers.js
protocolListController.js
protocolManager.js
sessionDesign.js

Verantwortung:

- Protokollerstellung
- Protokollspeicherung
- Protokollvalidierung

Monte Carlo

monteCarlo.js

monteCarloSummaryView.js

monteCarlo/

├─ monteCarloEngine.js

├─ monteCarloSampling.js

├─ monteCarloDiagnostics.js

├─ monteCarloStats.js

├─ monteCarloHistogram.js

└─ ...

Verantwortung:

- Simulation
- Sampling
- Clamp Analyse
- Diagnostik

Experiment

experiment.js

experiment/

- |— experimentRuntime.js
- |— experimentTrials.js
- |— experimentTargets.js
- |— experimentConditions.js
- |— experimentSummary.js
- |— ...

Verantwortung:

- Experimentdurchführung
- Trials
- Targets
- Ergebnisberechnung

4. Target Layer

Ordner:

static/javascript/targets

TargetFactory

Erzeugt Zielformen.

Target

Basisklasse.

TouchArea

Berechnet Trefferbereiche.

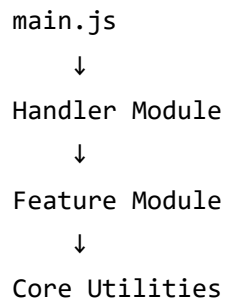
TargetDebugOverlay

Visualisiert Trefferbereiche.

Nur Debug-Modus.

Designprinzip

Nach dem Refactoring gilt:



Keine Fachlogik soll direkt in `main.js` liegen.